UPImesh: Architecture & Implementation Guide

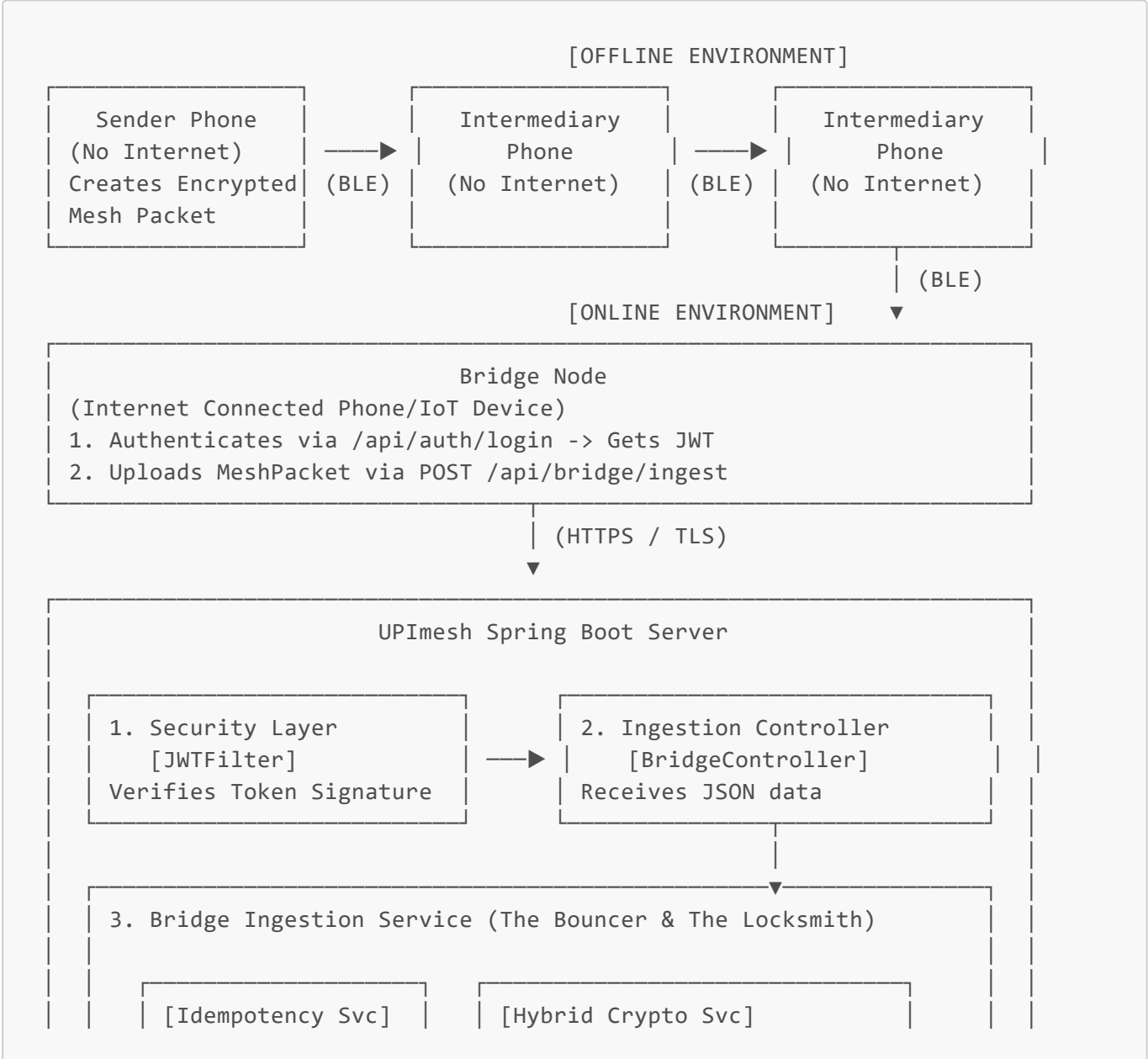
The Core Concept: "The Locked Box on a Mesh"

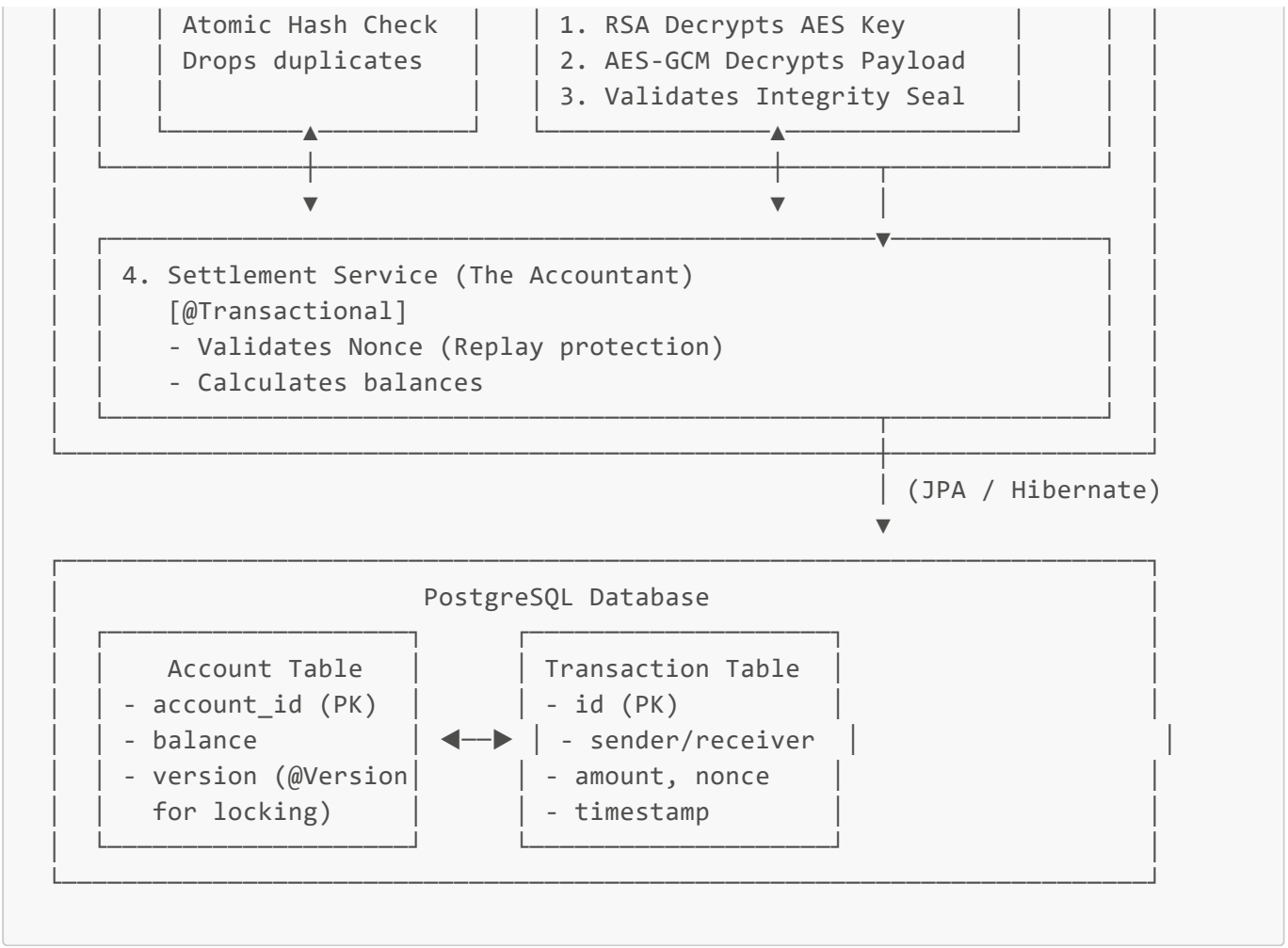
Imagine you are in a basement with zero internet connectivity. You need to pay a friend ₹500.

Because your phone cannot reach the bank, it creates a digital "IOU" (Payment Instruction). To stop anyone from snooping or changing the amount, your phone puts this IOU in a cryptographically secure "locked box" (Mesh Packet). Your phone then broadcasts this box to other offline phones nearby using Bluetooth. These phones act as a bucket brigade, passing the box around.

Eventually, one of these strangers walks out of the basement. Their phone connects to the 4G network and acts as a **Bridge Node**. It silently uploads the locked box to our Spring Boot server. The server unlocks it, checks the math, and finalizes the money transfer.

System Architecture Diagram





Solving the 3 Hard Problems of Offline Payments

To make this mesh network secure and reliable, the backend architecture was engineered specifically to solve three major challenges.

Problem 1: Snooping & Tampering

How do you stop the intermediary phones from reading your payment details or changing the amount from ₹500 to ₹5000?

Solution: Hybrid Cryptography & Integrity Seals We use a two-step cryptographic process to secure the payload before it ever leaves the sender's phone.

1. **AES-256-GCM (Symmetric):** This is our fast, heavy-duty safe. The offline phone puts the payment JSON into this safe and locks it with a randomly generated, one-time key. **GCM** (Galois/Counter Mode) provides an "Authentication Tag"—think of it as a tamper-evident wax seal. If an intermediary tries to alter the payment amount, the wax seal breaks. When our server tries to unlock it, the math will violently fail, and the packet is destroyed (**Data Integrity**).
2. **RSA-2048 (Asymmetric):** This is our super-secure padlock. We take the one-time AES key, put it in the RSA padlock, and lock it using the Server's Public Key. Only the server holds the Private Key needed to open it. Intermediaries see nothing but mathematical gibberish (**Data Confidentiality**).

Problem 2: The Duplicate Storm

If 5 people walk out of the basement at the exact same time, the server will receive 5 identical copies of your payment. How do we ensure you aren't charged 5 times?

Solution: In-Memory Idempotency & Atomic Hashing Idempotency is the ability to safely process the same request multiple times without changing the result. We *could* ask the database, "Have you seen this payment before?" But databases are slow. If 50 duplicates hit the server at the exact same millisecond, they might all bypass the database check simultaneously.

Instead, we use a `ConcurrentHashMap` directly inside the server's lightning-fast memory. We take a fingerprint (**SHA-256 Hash**) of the encrypted packet. We use a low-level JVM feature called an **Atomic Operation** (`putIfAbsent`). This mathematically guarantees that even if 50 requests try to save the fingerprint at the exact same nanosecond, only *one* thread will win. The other 49 instantly fail and drop the duplicate packet before any expensive decryption or database work occurs.

Problem 3: Replay Attacks

What if a malicious person captures your encrypted packet today, and tries to send it to the server again 3 weeks later to drain your account?

Solution: Freshness & Cryptographic Nonce To stop this, the sender's offline phone stamps two things inside the encrypted safe: a timestamp (`signedAt`) and a **Nonce** (Number Used Once). First, the server rejects any packet older than 24 hours. Second, when the server successfully processes a fresh payment, it permanently records the Nonce in the database. If a hacker tries to send the packet again later, the server checks the database, sees the used Nonce, and instantly blocks the attack.

The Supporting Infrastructure

To protect the system from the outside and ensure data is never lost on the inside, the architecture relies on two critical boundaries.

1. The Front Door: Stateless JWT Security

We only accept packets from approved Bridge Nodes to prevent spam and denial-of-wallet attacks. We use a **Stateless Architecture** powered by **JWT (JSON Web Tokens)**. When a packet arrives, the `JWTFilter` intercepts it. It uses cryptography to verify the mathematical signature on the token. If the signature is fake or missing, the door is slammed shut before the packet even reaches the ingestion service.

2. The Vault: Database Safety (ACID)

When dealing with money, database mistakes are unacceptable. We use PostgreSQL and rely on strict **ACID properties**.

- **Atomicity (`@Transactional`):** We wrap the whole settlement process into a single transaction. If the server loses power right after deducting the sender's money, the database automatically hits the "undo" button (**Rollback**). Money is never lost in limbo.
- **Race Conditions & Optimistic Locking (`@Version`):** What if Alice's phone sends two completely different payments while offline (buying coffee and paying rent), and both hit the server simultaneously? They might both read her balance as ₹1000, do their own math, and overwrite each other. We solve this using **Optimistic Locking**. Every account has a "Version Number." If two threads try to update the

balance at the same time, the first one succeeds and changes the version. The second thread realizes the version has changed, throws an error, and stops, preventing the system from accidentally creating or destroying money out of thin air.